



Haladó adatelemzési labor

Csapattagok: Szendrey Máté

Nagy Csaba Dániel

Sütő István

Konzulens: Tumay Ádám

Dokumentáció

Fog detekciós modell

YOLOv5s alapú fogazati objektumdetekciós modell fejlesztési folyamatát ismertet. A modell a PyTorch Lightning keretrendszert és Weights & Biases (W&B) monitorozást használ a hatékony tanítás és iteráció érdekében. A megközelítés szisztematikus validálásra és fokozatos optimalizálásra épül, lépéseit alább részletezem.

Az első fázis az alapvető infrastruktúra és tanítási hurok helyességének ellenőrzése. Ezt egy rendkívül kicsi, 16 képből álló részhalmazon végezzük, amely adat-leakage és egyéb alapvető problémák kimutatása mellett azt is biztosítja, hogy a modell képes-e memorizálni az adatokat. Sikeres overfit sanity check esetén a tanítási veszteség erősen csökken az epoch-ok során, ez így történt.

A következő lépés egy konzervatív hiperparaméter optimalizációs lépés volt. A konzervatív azt jelenti, hogy nem az összes adaton futott és nem a teljes tanítási epoch szám mellett; így végeredményként kapott paraméter konfigurációk közül ennek fényében nem a látszólag legjobb megoldás lett kiválasztva hanem egy ígéretes, stabil tanítással rendelkező paraméter halmaz. A hiperparaméter-keresés a tanulási ráta (lr) és súlycsökkenés (weight_decay) optimalizálására összpontosított, miközben az egyéb paramétereket rögzítve tartotta.

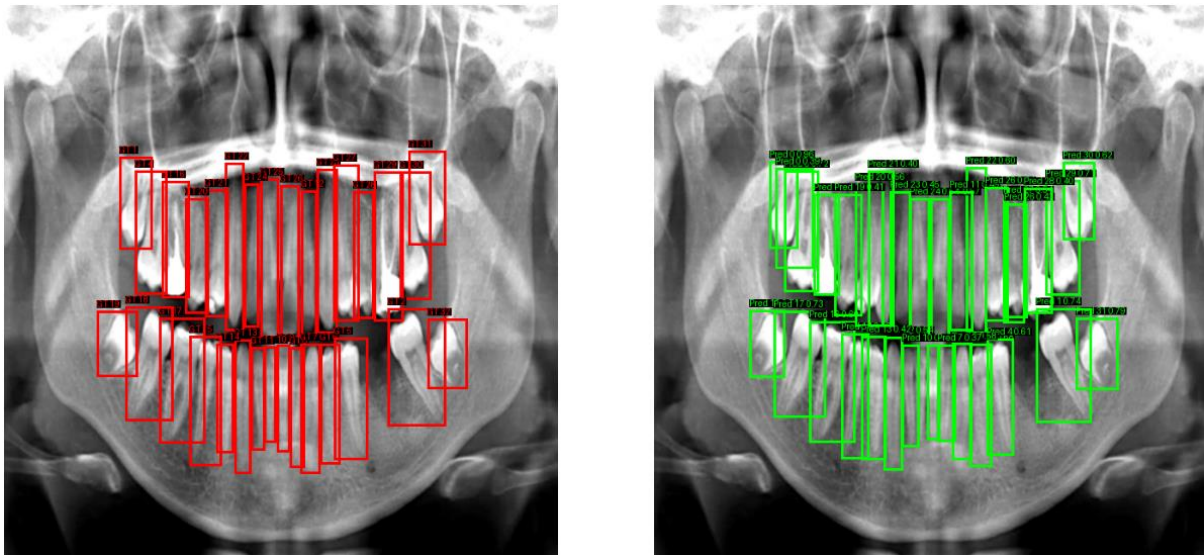
Harmadik fázisban a W&B sweep-ből nyert hiperparamétereket alkalmazva végezzük el a teljes tanítást az egész adathalmazon. Ez az igazi training futás, amely az éles modellt eredményezi. Az adatbetöltés során az összes adat a RoboFlow API-ból kerül letöltésre. A rekordok felosztása 70 százalék tanítási, 15 százalék validációs és 15 százalék test halmazra történik. Az augmentáció csak a tanítási halmazon alkalmazódik, amely véletlen flip, rotate, color jitter és egyéb geometriai transzformációkat tartalmaz. A validációs és test halmazok nem kerülnek augmentálásra az objektív kiértékelés biztosítása érdekében. A modell inicializálása az előtanított YOLOv5s checkpointból történik. Az előtanított súlyok kompatibilis állapota az anchor-okat kizárva kerül átvitelre az új modellbe. A loss konfigurációs hiperparaméterek az alábbiak: bounding box loss súly 0.05, osztály loss súly 0.3, objektivitás loss súly 0.7, osztály pozitív súly 1.0, objektivitás pozitív súly 1.0, focal loss gamma 0.0, label smoothing 0.0 és anchor target ratio 4.0.

Végül pedig a modell kiértékelése következett, majd a kimenetén különböző heurisztikus lépések végzése a jobb eredmény érdekében. Gyakran előfordult, hogy egy fog címkéjére több dobozt is rajzolt a modell eltérő helyekre a képen. Ezekből csak a legnagyobb konfidencia szintű tippet tartjuk meg adott címkére. Másik heurisztika azt az esetet kezeli, hogy a modell ugyanazt a fogat több osztályként (label) detektálja. Ez valószínűleg a limitált adatkészlet miatt fordult elő gyakran. Az algoritmus azonosítja azokat a dobozokat, amelyek jelentős átfedésben vannak, de különböző osztályokkal vannak jelölve. Ilyen esetekben csak a magasabb konfidencia szintű tippet tartjuk meg adott fogon.

A modell final test halmazon elért kvantitatív eredményei az alábbiak. A Mean Average Precision (mAP) az objektumdetekciós modellek teljesítményének mérőszáma, amely az átlagos pontosságot számítja ki az összes fogoszály között (ebben az esetben 32 fogoszály) különböző IoU küszöbök mellett, azaz minden osztályhoz külön kiszámítja a pontosságot,

majd ezeket átlagolja. A Mean Average Precision (mAP) átlagos értéke 0.3108, amely a szokásos detekciós modellekhez képest alacsonynak tűnik. Az $mAP@0.5$ (IoU threshold 0.5) érték 0.4940, amely jobb, azonban az $mAP@0.75$ (IoU threshold 0.75) érték 0.3577, amely jelentősen csökken. A Mean Average Recall (mAR) értékek következőképpen alakultak: mar_1 0.4612, mar_{10} 0.4612 és mar_{100} 0.4612, amely azonos marad a detekciók száma szerint.

Azonban fontos megjegyezni, hogy ezek a numerikus metrikák az alkalmazás szempontjából megtévesztőek. A kvalitatív vizuális kiértékelés szignifikánsan biztatóbb képet mutat. A modell az esetek nagy többségében helyesen detektálja az összes fogat és körülbelül jó dobozokat rajzol körülöttük. A numerikus mutatók azért alacsonyabbak, mivel a modell gyakran összekeveri az egyedi fog osztályozást. Az egyik fog detektálódhat különböző osztályokkal többszörösen vagy nem megfelelő osztályozással, amely az mAP metrikát negatívan befolyásolja.



Classification model

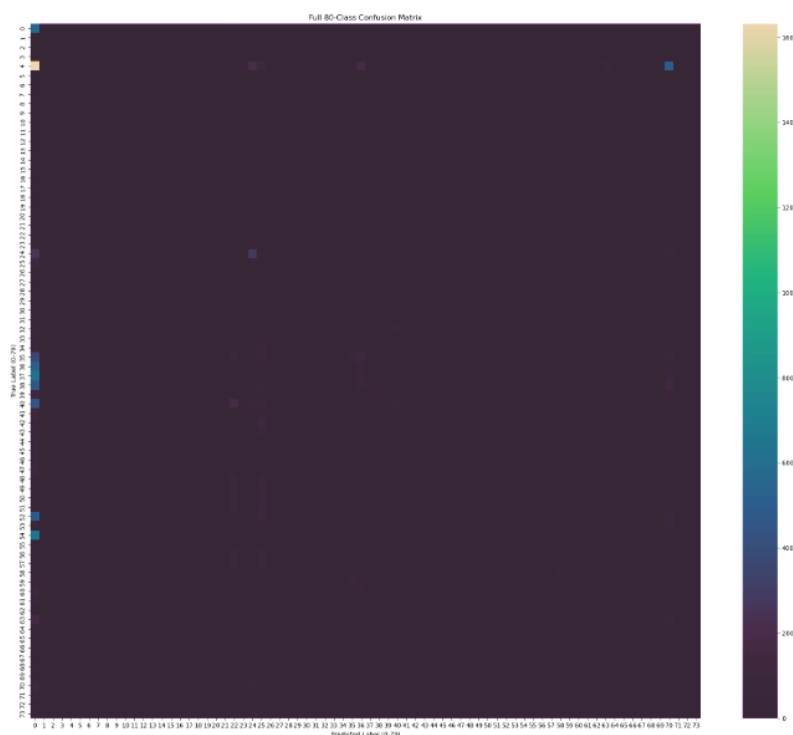
A teljes pipelineban ez a modul a második fázist képviseli. A bemeneti adatokat egy előzetes objektumdetektor modell, a YOLO szolgáltatja. A YOLO a teljes panorámaröntgenen lokalizálja a fogakat, a classification modellem pedig már kizárólag ezeket az egyenként, bounding box-ok alapján kivágott fog képeket kapja meg bemenetként. Ennek a szeparációnak köszönhetően a hálózathoz nem a teljes röntgen zajában kell keresgélnie, hanem dedikáltan az apró, 224x224 pixeles textúrák elemzésére tud fókuszálni.

A fejlesztés első szakaszában az adathalmaz eredeti, rendkívül részletes annotációit próbáltam felhasználni, ami összesen 80 különböző osztályt jelentett. Alapmodellként a ResNet-50 architektúrát választottam.

A betanítási kísérletek során azonban világossá vált, hogy ez nem vezetett megfelelő eredményre. A hálózat képtelen volt stabilan elkülöníteni a 80 kategóriát, aminek két fő oka volt:

1. **Class Imbalance:** Az egészséges fogak és a gyakori fogszámok túlnyomó többségben voltak, míg a specifikus, ritka elváltozások alig jelentek meg az adathalmazban.

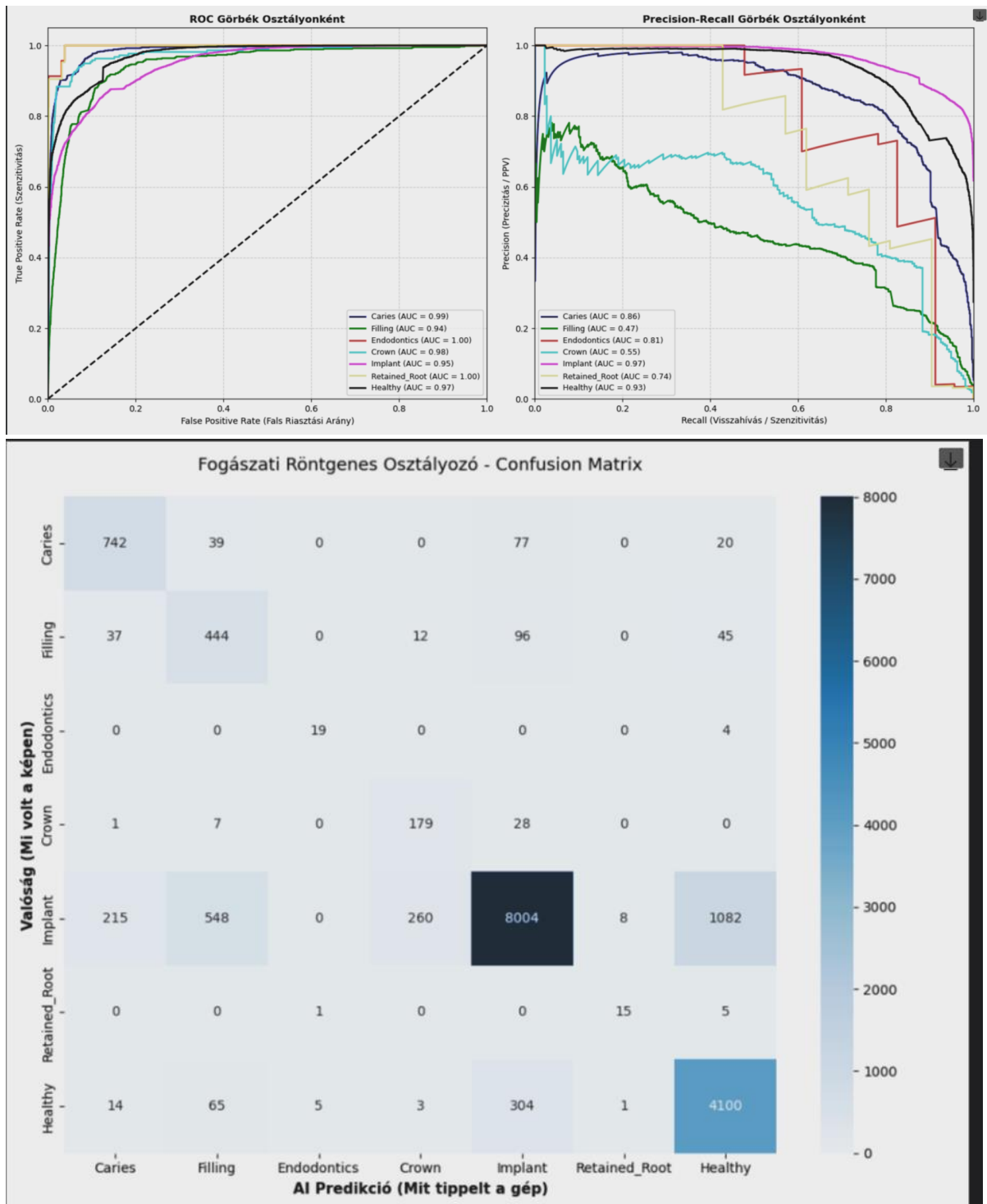
2. **Vizuális hasonlóság:** Az egyes fogszámok vizuálisan szinte azonos egészséges fogak közötti különbséget a modell nem tudta megbízhatóan leképezni. Ezek miatt a validációs hiba magas maradt, a modell konvergenciája pedig instabil volt.



Ezt követően változtatva lett néhány helyen. A paraméter igényes ResNet-50-et lecseréltem az EfficientNet-B0 architektúrára, amely modernebb skálázódásának köszönhetően sokkal hatékonyabban fókuszál a finom texturális különbségekre.

A legfontosabb döntés a 80 címke 7 kategóriába való összevonása volt. A kategóriákat úgy alakítottam ki, hogy klinikailag is jól elkülöníthetők legyenek (Tömés, Gyökérkezelt, Korona, Implantátum, Bentmaradt gyökér, Egészséges). Kiemelt szempont volt, hogy a 44-es kódú Fogszuvasodás (Caries) egy teljesen önálló csoportot alkosson, mivel ennek detektálása volt a projekt feladat célja.

Az osztályok bevezetése áttörést hozott. A kiértékelés során a Precision-Recall és a ROC görbék drasztikusan javultak. A Confusion Matrix letisztult, a predikciók a főátló mentén sűrűsödtek.



Ezután a modell javítása érdekében finomhangoltam még ezt a modellt. Alapként a 7-osztályos betanítás során létrejött legjobb F1 értékkel rendelkező modellt használtam, amely már kiváló reprezentációkkal rendelkezett a fogak anatómiájáról. A modell utolsó Linear kimeneti rétegét lecseréltem, és innentől kezdve kizárólag egy bináris osztályozási feladatra (Caries vs. No Caries) tanítottam tovább.

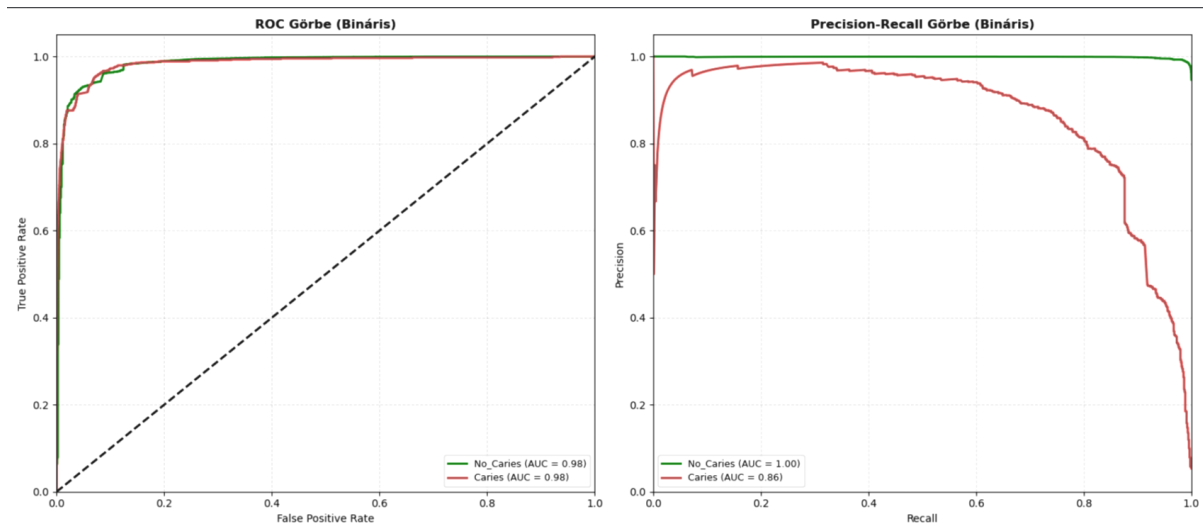
A finomhangolás során két kulcsfontosságú technikát alkalmaztam:

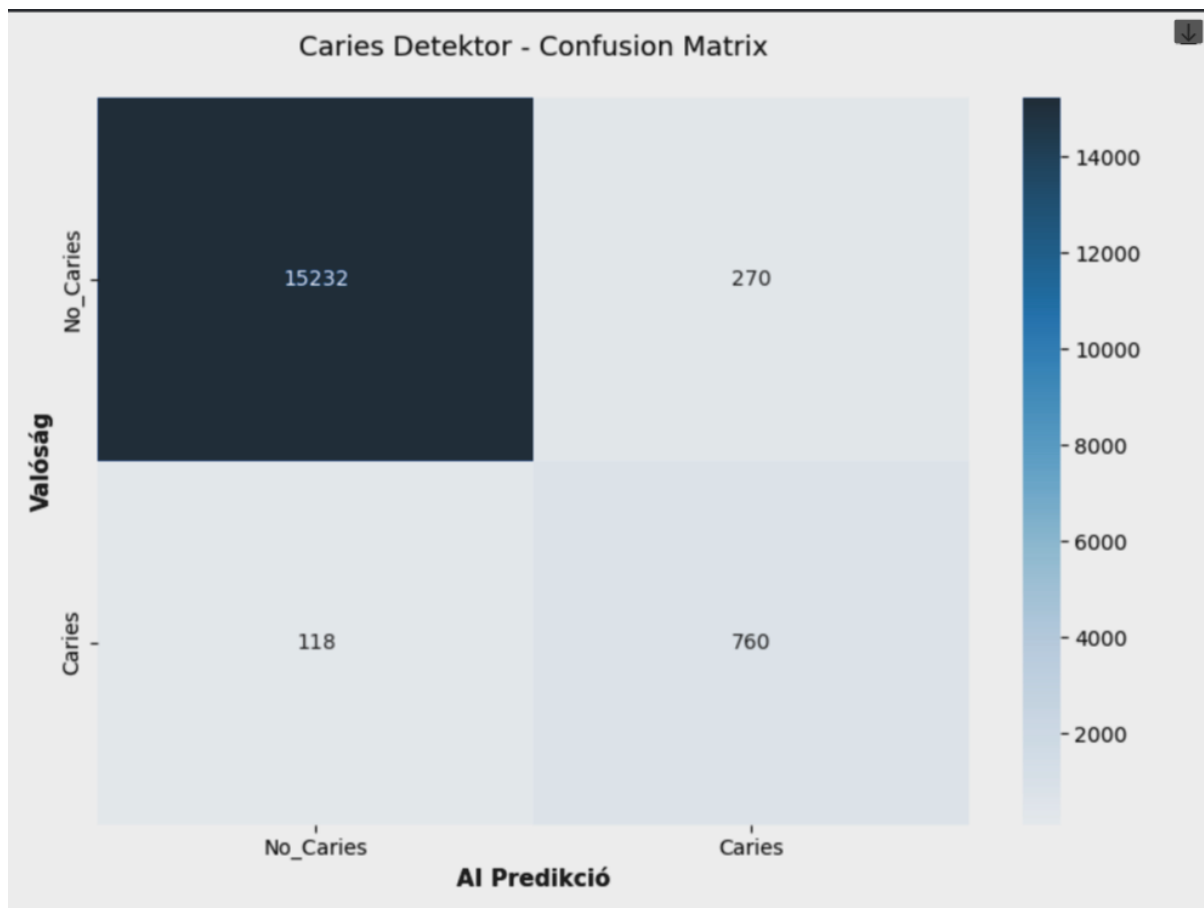
- Rendkívül alacsony tanulási rátát állítottam be, hogy a modell ne felejtse el a korábban megtanult anatómiai alaptudását.
- Class weights vezettem be a Loss függvényben. Mivel a szuvasodások aránya lényegesen kisebb volt az egészséges eseteknél, a súlyozás arra kényszerítette a hálózatot, hogy a ritkább Caries osztály eltévesztését sokkal szigorúbban büntesse.

Ezzel a lépéssel a modell sikeresen specializálódott a szuvasodások magas szenzitivitású felismerésére, habár nem javult ettől érzékelhetően a modell.

A kísérletek reprodukálhatóságához és a modell teljesítményének valós idejű monitorozásához a Weights & Biases (WandB) platformot integráltam a fejlesztői környezetbe. A tanítási ciklusok során folyamatosan logoltam a Train és Validation Loss értékeket, az Accuracy-t, valamint a kiegyensúlyozatlan adathalmazok legrelevánsabb mérőszámát, az F1-Score-t.

A rendszer kiértékeléséből jól látható, hogy a modell megfelelően elvégzi a feladatát. Kellő pontossággal szét tudja választani a szuvas és a nem szuvas fogakat is, figyelembe véve azt a tényt is, hogy jóval több nem szuvas fog szerepelt a tanító adatban, mint szuvas. A kiértékelés során vizuálisan megjelenítettem a Confusion Matrix-ot, a PR és a ROC görbét, valamint néhány metrikát is számolt a teszt rendszer.



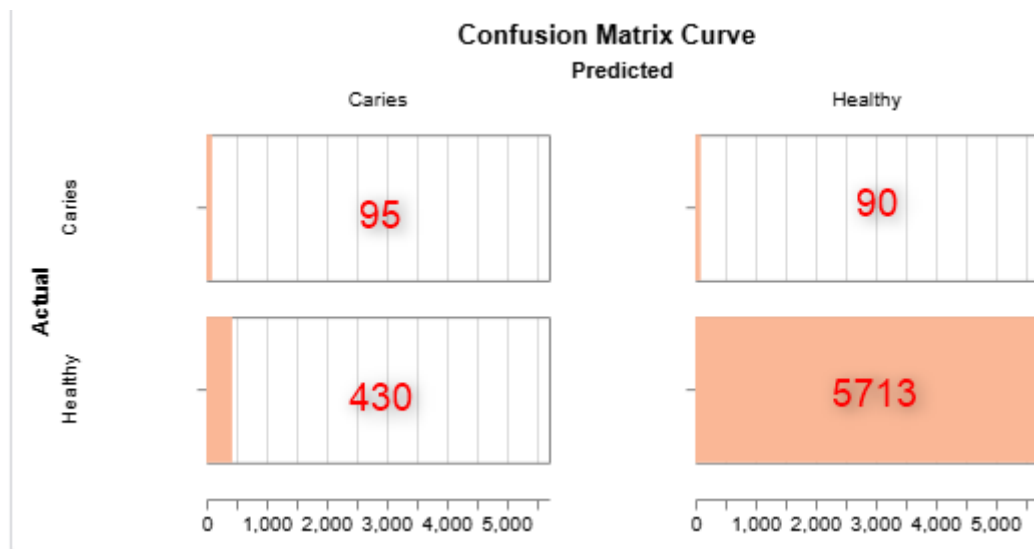


Teljes pipeline

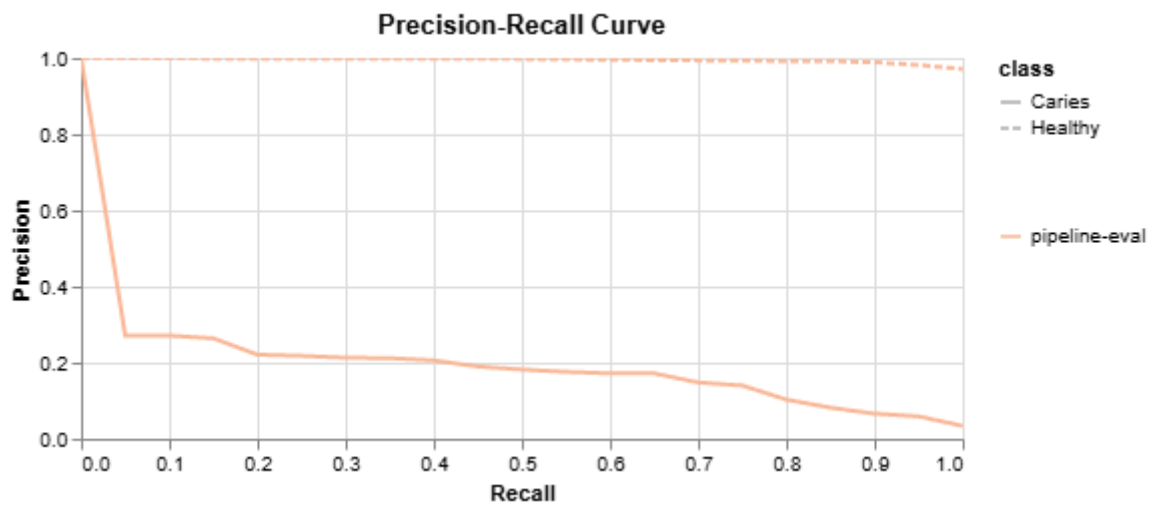
A teljes rendszer értékelésekor a bemenet a teljes panorámakép, a kimenet pedig az, hogy a rendszer mely fogakat jelöli szuvasnak. Ez a legfontosabb gyakorlati értékelési szint, mert itt együtt jelennek meg a detekciós és klasszifikációs hibák.

Az end-to-end (E2E) folyamat során a rendszer először egy nagyítás-és-kivágás (zoom-and-crop) eljárást alkalmaz a panorámaröntgeneken, majd a YOLOv5 modell segítségével detektálja a fogak határoló dobozait. A sikeresen megtalált fogakat a rendszer ezután kivágja, és egy EfficientNet-B0 alapú osztályozó hálózathoz adja át, amely eldönti, hogy az adott fog egészséges-e vagy szuvas.

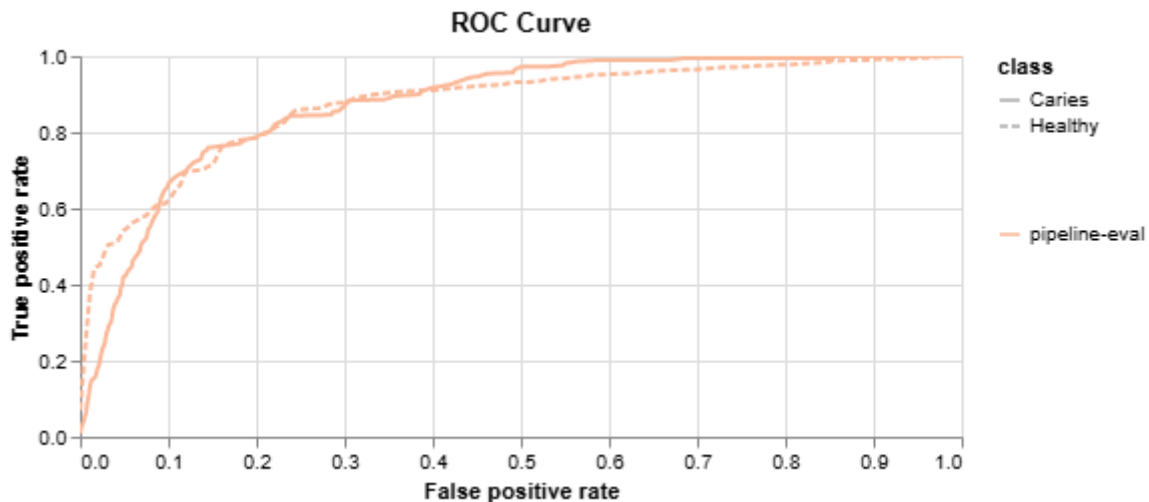
Teljesítmény



E2E konfúziós mátrix



E2E PR görbe



E2E ROC görbe

--- CLASSIFICATION METRICS (Matched Teeth Only) ---				
	precision	recall	f1-score	support
Healthy	0.98	0.93	0.96	6143
Caries	0.18	0.51	0.27	185
accuracy			0.92	6328
macro avg	0.58	0.72	0.61	6328
weighted avg	0.96	0.92	0.94	6328

E2E kiértékelési metrikák

Összességében a teljes rendszer pontossága (accuracy) 0.92 lett. Ez a látszólag magas érték nagyrészt az osztályok közötti egyensúlyhiánynak és a nagyszámú egészséges fog dominanciájának (6143 egészséges minta szemben a 185 szuval) köszönhető. A gyakorlati értékelés során tehát megállapítható, hogy a detekciós és klasszifikációs hibák együttes megjelenése leginkább a szuvas fogak felülreprezentált fals pozitív jelzéseiben (alacsony precizitás) csúcsosodik ki.

Kész alkalmazás

Az alkalmazás grafikus felület nélkül (CLI) működik. A futtatásához **Docker** környezet szükséges. A használat lépései:

1. A Docker kép fájl (image) elkészítése

Az alábbi parancsot a gyökérkönyvtárban a Docker kép fájl felépítéséhez:

`docker build -t caries-detection .`

2. A Konténer futtatása

A helyi input (bemenet), output (kimenet) és log (napló) mappákat a konténerhez kell csatolni, meg kell a Weights & Biases API kulcsát (amennyiben használja), majd futtatni a képfájlt. A szabványos kimenet és a hibaüzenetek egy **run.log** nevű fájlba lesznek átirányítva.

```
docker run --rm --gpus all -v "[path/to/input]:/work/input" ^  
-v "[path/to/output]:/work/output" -v "[path/to/log]:/work/log" ^  
-e WANDB_API_KEY="[wandb_api_key]" caries-detection > "[path/to/log]\run.log" 2>&1
```

3. A Naplófájlok (logok) megtekintése

Várd meg, amíg a folyamat befejeződik. A megadott log mappán belül az alábbiakat találhatók:

- **run.log:** A Docker futtatásának teljes naplóját tartalmazza, a csomagok importálásától egészen a folyamat legvégéig.
- **pipeline.log:** Csak a Python folyamat specifikus végrehajtási naplóit tartalmazza (pl. a modellek betöltését, a képfeldolgozási lépéseket).

Futtatási paraméterek (Parancssori Argumentumok)

A rendszer működését az alábbi parancssori kapcsolók (flagek) segítségével személyre lehet szabni.

Könyvtárak (elérési utak)

- **--input_dir:** A feldolgozandó nyers röntgenképeket tartalmazó bemeneti könyvtár. *(Alapértelmezett: /work/input)*
- **--output_dir:** Az a könyvtár, ahová a feldolgozás eredményei (annotált .jpg képek és részletes .json bounding box-ok) mentésre kerülnek. *(Alapértelmezett: /work/output)*
- **--log_dir:** A folyamat során generált szöveges naplófájlok (pl. pipeline.log) mentési helye. *(Alapértelmezett: /work/log)*

Modell súlyok

- **--yolo_weights:** A YOLO fogfelismerő modell (határoló dobozok) súlyfájljának pontos elérési útja. *(Alapértelmezett: /work/models/detection.pt)*
- **--clf_weights:** Az EfficientNet osztályozó modell (fogszuvasodás felismerés) súlyfájljának pontos elérési útja. *(Alapértelmezett: /work/models/classification.ckpt)*

Bemenet

Ezek a kapcsolók vezérlik, hogy milyen előfeldolgozási lépések fussanak le a bemeneti képeken.

- **--use_zoom**: Bekapcsolása esetén a program egy nagyítás-és-kivágás (zoom-and-crop) heurisztikát alkalmaz a bemeneti képeken az objektumfelismerés (object detection) futtatása előtt.

Kimenet

Ezek a kapcsolók vezérlik, hogy mi kerüljön felrajzolásra a végső kimeneti képekre.

- **--draw_healthy**: Bekapcsolása esetén a program egy **zöld dobozt** rajzol az egészségesnek minősített fogak köré.
- **--draw_caries**: Bekapcsolása esetén a program egy **piros dobozt** rajzol a fogszuvasodásosnak (betegnek) minősített fogak köré.

! Ha egyik vizualizációs kapcsolót sincs megadva a szkript futtatásakor, a program automatikusan **mindkét kategóriát (zöld és piros dobozok)** felrajzolja a képekre. Ha kifejezetten csak az egyik típust szeretnénk látni, akkor csak azt az egy kapcsolót kell megadni meg. !

Naplózás és monitorozás (Weights & Biases)

- **--use_wandb**: Engedélyezi a felhőalapú naplózást a Weights & Biases (WandB) rendszerén keresztül, amely feltölti a feldolgozott képeket és az alapvető statisztikákat. Ennek használatához a Docker konténer futtatásakor környezeti változóként meg kell adni a WANDB_API_KEY kulcsot!
- **--wandb_project**: Megadja annak a WandB projektnek a nevét, ahová a futtatás eredményei szinkronizálva lesznek. *(Alapértelmezett: tooth-e2e-pipeline)*

Használati példa

Egy példa a folyamat futtatására abban az esetben, ha **csak a fogszuvasodást jelző piros dobozokat** szeretnénk felrajzolni, engedélyezni szeretnénk a **WandB naplózást**, és a futtatást egy my-dental-project nevű egyéni projekthez szeretnénk rendelni.

(Az az egyéni argumentumokat a parancs legvégére kell hozzáadni, közvetlenül a caries-detection után, de még a napló átirányítását jelző > jel előtt):

```
docker run --rm --gpus all -v "D:\projects\dental\input:/work/input" ^
-v "D:\projects\dental\output:/work/output" -v "D:\projects\dental\log:/work/log" ^
-e WANDB_API_KEY="abc123yourkeyhere" ^
caries-detection --draw_caries --use_wandb --wandb_project "my-dental-project" ^
> "D:\projects\dental\log\run.log" 2>&1
```

